

EDS Lab Assignments

Practical 2

Name: Keshav Devaji Bahekar

PRN: 202501040031

3.1.1 Numpy Array Operations

Problem Statement:

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Code:

```
import numpy as np
rows, cols = map(int, input().split())
elements = []

for _ in range(rows):
    elements.extend(map(int, input().split()))

array = np.array(elements).reshape(rows, cols)
print(array)
print(array.ndim)
print(array.shape)
print(array.size)
```

Output:

CODETANTRA

Home

Learn Anywhere

202501040043@mitaoe.ac.in

Support

Logout

3.1.1. Numpy Array Operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array
- A tuple representing the shape of the array
- An integer representing the total number of elements in the array

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

numpyarr...

```
1 import numpy as np
2 rows, cols = map(int, input().split())
3 elements = []
4
5 for _ in range(rows):
6     elements.extend(map(int, input().split()))
7
8 array = np.array(elements).reshape(rows, cols)
9 print(array)
10 print(array.ndim)
11 print(array.shape)
12 print(array.size)
```

Average time

0.016 s

15.92 ms

Maximum time

0.025 s

25.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 1

Expected output

3 4

1 2 3 4

5 6 7 8

9 10 11 12

[[1 2 3 4]

[5 6 7 8]

[9 10 11 12]]

2

(3, 4)

12

Actual output

3 4

1 2 3 4

5 6 7 8

9 10 11 12

[[1 2 3 4]

[5 6 7 8]

[9 10 11 12]]

2

(3, 4)

12

Test case 2

Terminal

Test cases

< Prev

Next >

3.2.1 Numpy: Matrix Operations

Problem Statement:

The given code takes two 3×3 matrices, matrix_a, and matrix_b, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** ($\text{matrix_a} + \text{matrix_b}$)
2. **Subtraction** ($\text{matrix_a} - \text{matrix_b}$)
3. **Element-wise Multiplication** ($\text{matrix_a} * \text{matrix_b}$)
4. **Matrix Multiplication** ($\text{matrix_a} \cdot \text{matrix_b}$)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Code:

```
import numpy as np

# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# Addition
addition_result = matrix_a + matrix_b
print("Addition (A + B):")
```

```
print(addition_result)
```

```
# Subtraction
```

```
subtraction_result = matrix_a - matrix_b
```

```
print("Subtraction (A - B):")
```

```
print(subtraction_result)
```

```
# Multiplication (element-wise)
```

```
elementwise_multiplication_result = matrix_a * matrix_b
```

```
print("Element-wise Multiplication (A * B):")
```

```
print(elementwise_multiplication_result)
```

```
# Matrix multiplication (dot product)
```

```
matrix_multiplication_result = np.dot(matrix_a, matrix_b)
```

```
print("A dot B:")
```

```
print(matrix_multiplication_result)
```

```
# Transpose
```

```
transpose_a = matrix_a.T
```

```
print("Transpose of A:")
```

```
print(transpose_a)
```

Output:

The screenshot displays the CodeTANTRA web interface. On the left, the task description for '3.2.1. NumPy: Matrix Operations' is shown, detailing the required operations: Addition, Subtraction, Element-wise Multiplication, Matrix Multiplication, and Transpose of Matrix A. It also specifies the input and output formats. The main area on the right shows the test results for two test cases. Test case 1 is passed, and Test case 2 is also passed. The interface includes a 'Debugger' button and a 'Test' button. The test results are displayed in a table with columns for 'Expected output' and 'Actual output'.

Test Case	Expected output	Actual output
Test case 1	Enter Matrix A: [[2 3 4] [5 6 7] [8 9 10]] Enter Matrix B: [[1 2 3] [4 5 6] [7 8 9]] Addition (A + B): [[3 5 7] [9 11 13] [15 17 19]] Subtraction (A - B): [[1 1 1] [1 1 1] [1 1 1]] Element-wise Multiplication (A * B): [[2 6 12] [20 30 42] [56 72 90]] A dot B: [[14 32 22] [32 20 34] [44 34 46]] Transpose of A: [[2 5 8] [3 6 9] [4 7 10]]	Enter Matrix A: [[2 3 4] [5 6 7] [8 9 10]] Enter Matrix B: [[1 2 3] [4 5 6] [7 8 9]] Addition (A + B): [[3 5 7] [9 11 13] [15 17 19]] Subtraction (A - B): [[1 1 1] [1 1 1] [1 1 1]] Element-wise Multiplication (A * B): [[2 6 12] [20 30 42] [56 72 90]] A dot B: [[14 32 22] [32 20 34] [44 34 46]] Transpose of A: [[2 5 8] [3 6 9] [4 7 10]]
Test case 2		

3.2.2 Numpy: Horizontal and Vertical Stacking of Arrays

Problem Statement:

You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Code:

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
a = np.hstack((arr1, arr2))
print("Horizontal Stack:")
print(a)

# Perform vertical stacking (vstack)
b = np.vstack((arr1, arr2))
print("Vertical Stack:")
print(b)
```

Output:

CODETANTRA

HomeLearn Anywhere

202501040043@mitaoe.ac.inSupportLogout

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

100%

🔍🔧📄🔖

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking**: Stack the two matrices horizontally (side by side).
- **Vertical Stacking**: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

1import numpy as np

2

Average time0.030 s30.20 ms

Maximum time0.036 s36.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Enter Array1:

1 2 3

4 5 6

7 8 9

Enter Array2:

4 5 6

7 8 9

10 11 12

Horizontal Stack:

[[1 2 3 4 5 6]

[4 5 6 7 8 9]

[7 8 9 10 11 12]]

Vertical Stack:

[[1 2 3]

[4 5 6]

[7 8 9]

[4 5 6]

[7 8 9]

[10 11 12]]

Actual output

Enter Array1:

1 2 3

4 5 6

7 8 9

Enter Array2:

4 5 6

7 8 9

10 11 12

Horizontal Stack:

[[1 2 3 4 5 6]

[4 5 6 7 8 9]

[7 8 9 10 11 12]]

Vertical Stack:

[[1 2 3]

[4 5 6]

[7 8 9]

[4 5 6]

[7 8 9]

[10 11 12]]

Test case 2

Terminal

Test cases

< Prev

Next >

3.2.3 Numpy: Custom Sequence Generation

Problem Statement:

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Code:

```
import numpy as np
# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())
# Generate the sequence using np.arange()
a = np.arange(start, stop, step)
# Print the generated sequence
print(a)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement and input/output formats are visible. The main editor shows the Python code for generating a sequence using `np.arange()`. The code takes three integer inputs (start, stop, step) and prints the resulting array. The terminal at the bottom shows the execution output for two test cases. Test case 1 with inputs 3, 5, and 7 results in the array `[3 5 7 9]`. Test case 2 with inputs 18, 28, and 10 results in the array `[18 28 38]`. The test runner indicates that both test cases passed.

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9 a = np.arange(start, stop, step)
10 # Print the generated sequence
11 print(a)
```

Test case 1: Expected output [3 5 7 9], Actual output [3 5 7 9]. Test case 2: Expected output [18 28 38], Actual output [18 28 38].

3.2.4 Numpy: Arithmetic and Statistical Operations, Mathematical Operations and Bitwise Operators

Problem Statement:

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Code:

```
import numpy as np
```

```
def array_operations(A, B):
```

```
    # Convert A and B to NumPy arrays
```

```
    A = np.array(A)
```

```
    B = np.array(B)
```

```
    # Arithmetic Operations
```

```
    sum_result = A + B
```

```
    diff_result = A - B
```

```
    prod_result = A * B
```

```
    # Statistical Operations
```

```
    mean_A = np.mean(A)
```

```
    median_A = np.median(A)
```

```
    std_dev_A = np.std(A)
```


Bitwise Operations

and_result = A & B

or_result = A | B

xor_result = A ^ B

Output results with one space between each element

print("Element-wise Sum:", ' '.join(map(str, sum_result)))

print("Element-wise Difference:", ' '.join(map(str, diff_result)))

print("Element-wise Product:", ' '.join(map(str, prod_result)))

print(f"Mean of A: {mean_A}")

print(f"Median of A: {median_A}")

print(f"Standard Deviation of A: {std_dev_A}")

print("Bitwise AND:", ' '.join(map(str, and_result)))

print("Bitwise OR:", ' '.join(map(str, or_result)))

print("Bitwise XOR:", ' '.join(map(str, xor_result)))

A = list(map(int, input().split())) # Elements of array A

B = list(map(int, input().split())) # Elements of array B

array_operations(A, B)

Output:

The screenshot displays a CodeTANTRA IDE interface. On the left, the problem statement for '3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators' is visible. It describes the task of completing the `array_operations` function, which takes two arrays `A` and `B` and performs various operations. The input format specifies two lines of space-separated integers. The output format requires printing the results of arithmetic, statistical, and bitwise operations in a specific format.

The main editor shows the following Python code:

```
9 # Arithmetic Operations
10 sum_result = A + B
11 diff_result = A - B
12 prod_result = A * B
13
14 # Statistical Operations
15 mean_A = np.mean(A)
16 median_A = np.median(A)
17 std_dev_A = np.std(A)
18
19 # Bitwise Operations
```

Below the code, the execution results are shown. The 'Test case 1' section indicates that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The 'Expected output' and 'Actual output' sections show the following results:

Expected output	Actual output
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
Element-wise Sum: 6 8 10 12	Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4	Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32	Element-wise Product: 5 12 21 32
Mean of A: 2.5	Mean of A: 2.5
Median of A: 2.5	Median of A: 2.5
Standard Deviation of A: 1.118033988749895	Standard Deviation of A: 1.118033988749895
Bitwise AND: 1 2 3 0	Bitwise AND: 1 2 3 0
Bitwise OR: 5 6 7 12	Bitwise OR: 5 6 7 12
Bitwise XOR: 4 4 4 12	Bitwise XOR: 4 4 4 12

3.2.5 Numpy: Copying and Viewing Arrays

Problem Statement:

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original_array and assigning it to view_array.
- Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:
Original array after modifying view: <original_array>
View array: <view_array>
- After modifying the copy:
Original array after modifying copy: <original_array>
Copy array: <copy_array>

Code:

```
import numpy as np
inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
```

```
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

Output:

The screenshot displays a coding challenge on the CodeTANTRA platform. The challenge is titled "3.2.5. Numpy: Copying and Viewing Arrays". The instructions state: "The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by: • Creating a view of the original_array and assigning it to view_array. • Creating a copy of the original_array and assigning it to copy_array." It also notes: "After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not."

Input Format:
• A single line of space-separated integers.

Output Format:
• After modifying the view:
Original array after modifying view: original_array
View array: view_array
• After modifying the copy:
Original array after modifying copy: original_array
Copy array: copy_array

Sample Test Cases

The code editor shows the following Python code:

```
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
12 copy_array = original_array.copy()
13
14 # Modify the view
15 view_array[0] = 99
16 print("Original array after modifying view:", original_array)
```

The execution results show that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The average time is 0.011 s and the maximum time is 0.014 s. The test case output is as follows:

Test case 1	Expected output	Actual output
1	99 28 38 48 58 68 78 88	99 28 38 48 58 68 78 88
2	Original array after modifying view: [99 28 38 48 58 68 78 88]	Original array after modifying view: [99 28 38 48 58 68 78 88]
3	View array: [99 28 38 48 58 68 78 88]	View array: [99 28 38 48 58 68 78 88]
4	Original array after modifying copy: [99 28 38 48 58 68 78 88]	Original array after modifying copy: [99 28 38 48 58 68 78 88]
5	Copy array: [18 88 38 48 58 68 78 88]	Copy array: [18 88 38 48 58 68 78 88]

3.2.6 Searching, Sorting, Counting, Broadcasting

Problem Statement:

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- 1. Searching:** Find the indices where `search_value` appears in `array1` and print these indices.
- 2. Counting:** Count how many times `count_value` appears in `array1` and print the count.
- 3. Broadcasting:** Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
- 4. Sorting:** Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `array1`.
2. The count of occurrences of `count_value` in `array1`.
3. The array after adding the `broadcast_value` to each element.
4. The sorted array.

Code:

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split())))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))
```

```
# Find indices where value matches in array1
a = np.where(array1 == search_value)[0]
print(a)
```

```
# Count occurrences in array1
b = np.count_nonzero(array1 == count_value)
print(b)
```

```
# Broadcasting addition
c = array1 + broadcast_value
print(c)
```

```
# Sort the first array
d = np.sort(array1)
print(d)
```

Output:

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user. Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- 1. Searching**: Find the indices where `search_value` appears in `array1` and print these indices.
- 2. Counting**: Count how many times `count_value` appears in `array1` and print the count.
- 3. Broadcasting**: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
- 4. Sorting**: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

1. The indices where `search_value` occurs in `array1`.
2. The count of occurrences of `count_value` in `array1`.
3. The array after adding the `broadcast_value` to each element.
4. The sorted array.

Sample Test Cases

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 a = np.where(array1 == search_value)[0]
13 print(a)
14
15 # Count occurrences in array1
16 b = np.count_nonzero(array1 == count_value)
```

Test Results:

- Average time: 0.029 s
- Maximum time: 0.038 s
- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case 1:

Expected output	Actual output
1 1 2 2 2	1 1 2 2 2
Value to search: 1	Value to search: 1
Value to count: 2	Value to count: 2
Value to add: 2	Value to add: 2
[0 1 2]	[0 1 2]
3	3
[3 3 3 4 4 4]	[3 3 3 4 4 4]
[1 1 1 2 2 2]	[1 1 1 2 2 2]

3.2.7 Student Data Analysis and Operations

Problem Statement:

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject:** Count the number of students who scored 90 or more marks in each subject.

- **Find the count of subjects in which each student scored ≥ 90 :** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Code:

```
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

roll = a[:, 0].astype(float)
s1 = a[:, 1]
s2 = a[:, 2]
s3 = a[:, 3]

# 1. Print all student details
print("All student Details:\n", a)

# 2. print total students
print("Total Students:", len(a))

# 3. Print all student Roll numbers
print("All Student Roll Nos", roll)

# 4. Print subject 1 marks
print("Subject 1 Marks", s1)

# 5. print minimum marks of Subject 2
print("Min marks in Subject 2", np.min(s2))

# 6. print maximum marks of Subject 3
print("Max marks in Subject 3", np.max(s3))

# 7. Print All subject marks
print("All subject marks:", a[:, 1:])

# 8. print Total marks of students
total = s1 + s2 + s3
print("Total Marks", total)

# 9. print average marks of each student
```

```

print(np.round(total/3, 1))

# 10. print average marks of each subject
print("Average Marks of each subject", np.mean(a[:, 1:], axis = 0))

# 11. print average marks of S1 and S2
print("Average Marks of S1 and S2", np.mean([s1, s2], axis = 1))

# 12. print average marks of S1 and S3
print("Average Marks of S1 and S3", np.mean([s1, s3], axis = 1))

# 13. print Roll number who got maximum marks in Subject 3
print("Roll no who got maximum marks in Subject 3", float(roll[np.argmax(s3)]))

# 14. print Roll number who got minimum marks in Subject 2
print("Roll no who got minimum marks in Subject 2", float(roll[np.argmin(s2)]))

# 15. print Roll number who got 24 marks in Subject 2
print("Roll no who got 24 marks in Subject 2", roll[s2 == 24].astype(float).reshape(1, -1))

# 16. print count of students who got marks in Subject 1 < 40
print("Count of students who got marks in Subject 1 < 40", np.sum(s1 < 40))

# 17. print count of students who got marks in Subject 2 > 90
print("Count of students who got marks in Subject 2 > 90:", np.sum(s2 > 90))

# 18. print count of students in each subject who got marks >= 90
print("Count of students in each subject who got marks >= 90:", np.sum(a[:, 1:] >= 90,
axis = 0))

# 19. print count of subjects in which each student got marks >= 90
print("Roll no:", roll)
print("Count of subjects in which student got marks >= 90:", np.sum(a[:, 1:] >= 90, axis =
1))

# 20. Print S1 marks in ascending order
print(np.sort(s1))

# 21. Print S1 marks >= 50 and <= 90
print(a[(s1 >= 50) & (s1 <= 90)].astype(float))
print(a.astype(float))

# 22. Print the index position of marks 79
print(np.where(s1 == 79))

```


